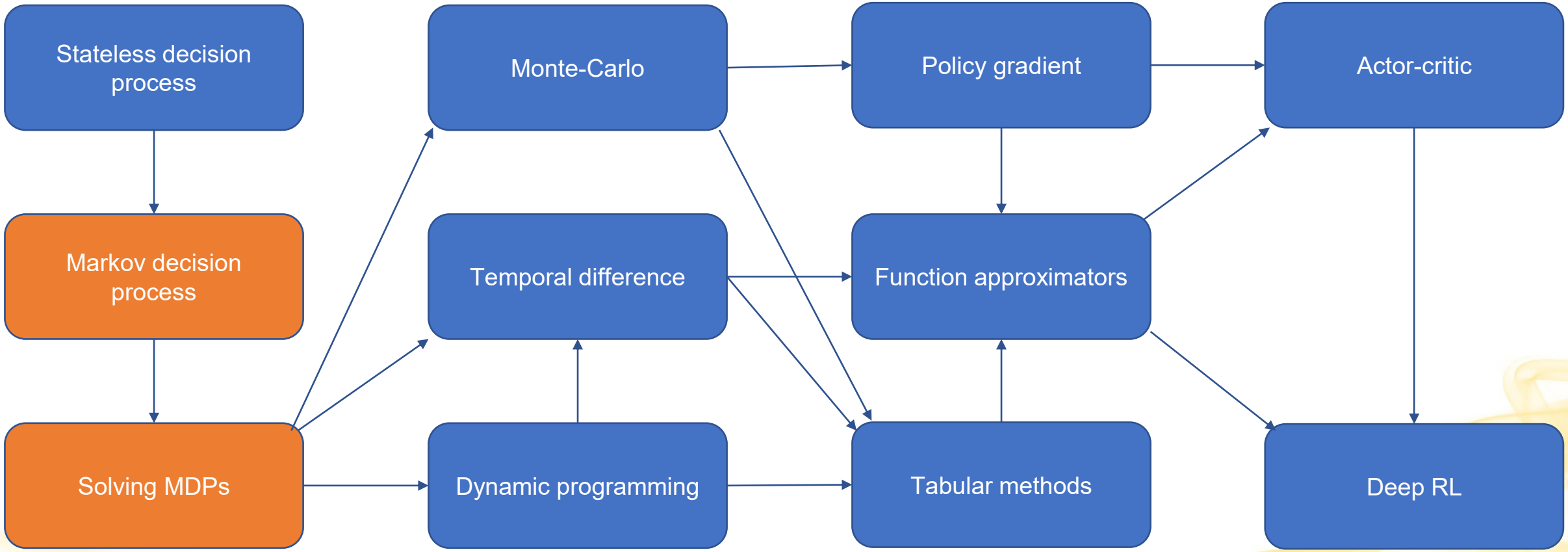
MARKOV DECISION PROCESSES

MSAI/MSML 635: Reinforcement Learning

© 2020, UNIVERSITY OF MARYLAND

The Roadmap



© 2020, UNIVERSITY OF MARYLAND

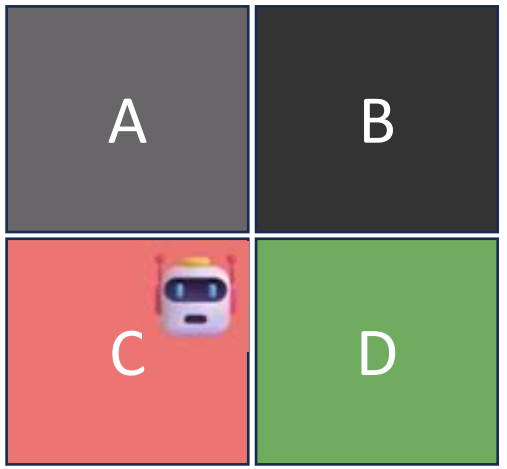
Markov Decision Processes (MDPs)

- MDPs formally describe an environment for reinforcement learning where the environment is fully observable
- The current state completely characterizes the process
- Almost all RL problems can be formulated as MDPs, e.g.
  - Optimal control primarily deals with continuous MDPs
  - Partially observable problems can be converted into MDPs
  - Bandits are MDPs with one state

© 2020, UNIVERSITY OF MARYLAND

Example: Grid World

- We will use a simple 2x2 grid to build intuition
- From states to rewards to Bellman equations and policy iteration
- This idea will scale from toy mazes to autonomous systems and games
- They establish the conceptual foundation for reinforcement learning
- The agent lives in a grid-world and walls block the agent's path
- Objective is to maximize cumulative reward



© 2020, UNIVERSITY OF MARYLAND

States

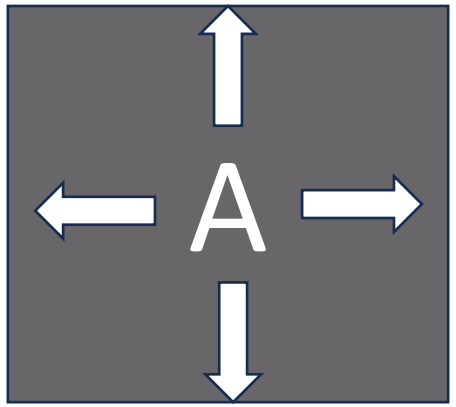
- A state describes the agent's current situation
- In the grid world, a state is simply the current square
- The set of all possible states is the state space {A, B, C, D}
- It is the answer to “Where am I right now?”
- For an autonomous vehicle the state is way more complex



© 2020, UNIVERSITY OF MARYLAND

Actions

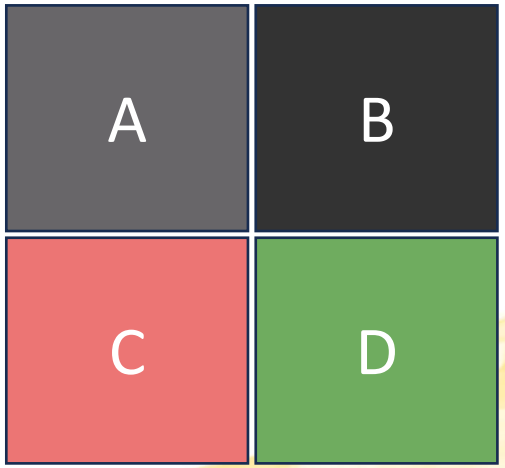
- Actions are the choices available to the agent
- Grid-world action space {North, South, East, West}
- The action space describes what the agent can do from a state
- It is the answer to “What can I do here?”



© 2020, UNIVERSITY OF MARYLAND

Rewards

- A reward is numerical feedback from the environment
- Positive rewards encourage behavior
- Negative rewards discourage behavior
- Rewards define what the agent is trying to optimize
- In grid-world
  - Reaching the goal (D): +10
  - Falling in the Pit (C): -10
  - Every other move: -1 (cost for wasting time)



© 2020, UNIVERSITY OF MARYLAND

Markov Property

- The future is independent of the past given the present
- Formally, a state S_t is Markov if an only if

$$P[S_{t+1} \mid S_t] = P[S_{t+1} \mid S_1, \ldots, S_t]$$

- The state captures all relevant information from the history
- Once the state is known, the history may be thrown away
- i.e., future depends on the present state and action—not the entire history
- This memoryless assumption makes the problem tractable

© 2020, UNIVERSITY OF MARYLAND

Why can History Explode?

- If decisions depended on every past event, the number of possible histories could become enormous
- Learning a separate rule for every history is impractical
- The Markov property collapses histories into states

© 2020, UNIVERSITY OF MARYLAND

What Happens Next? (Transitions)

- MDPs
  - Agent acts in an environment
  - The environment includes a set of possible states
  - At each state, a different set of actions might be available
  - Each state, action pair have a different expected outcome (reward and next state)
  - Transition probabilities describe the environment's dynamics

© 2020, UNIVERSITY OF MARYLAND

State Transition Matrix

- For a Markov state s and successor state s', the state transition probability is defined by

$$P_{ss'} = \mathbb{P}[S_{t+1} = s' | S_t = s]$$

- State transition matrix P defines transition probabilities form all states s to all successor states s',

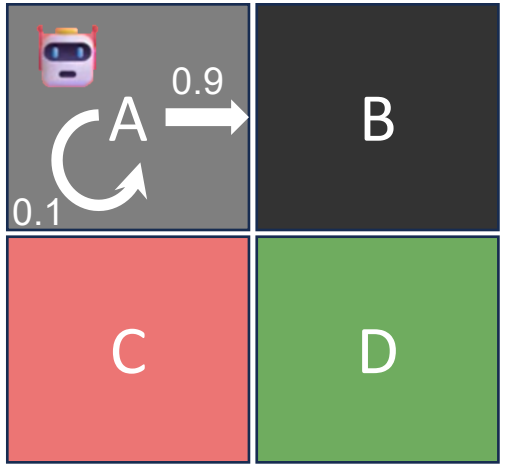
$$P = \begin{matrix} \\ \text{from} \end{matrix} \overset{\text{to}}{\begin{bmatrix} P_{11} & \cdots & P_{1n} \\ \vdots & \ddots & \vdots \\ P_{n1} & \cdots & P_{nn} \end{bmatrix}}$$

where each row of the matrix sums to 1.

© 2020, UNIVERSITY OF MARYLAND

Stochastic Transitions

- Real environments can be uncertain
- Example: moving east might reach B with probability 0.9
- With probability 0.1, the agent might remain in A
- Stochasticity makes planning probabilistic



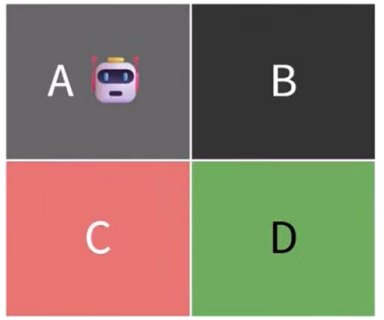
© 2020, UNIVERSITY OF MARYLAND

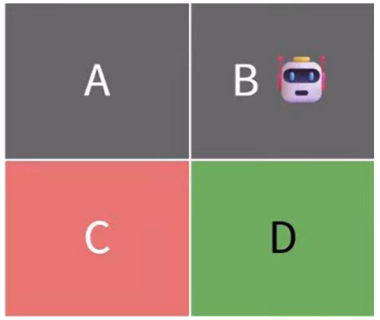
Trajectories

- A trajectory is the sequence of states, actions, and rewards
  - [State -> Action -> Reward -> State] -> Action -> Reward …
- It records the agent's interaction history
- Learning aims to generate trajectories with high return

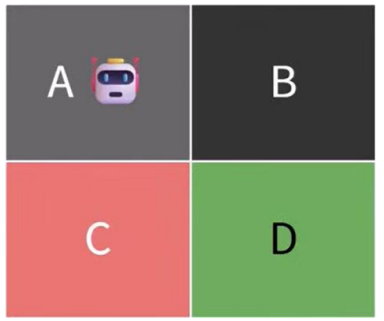
© 2020, UNIVERSITY OF MARYLAND

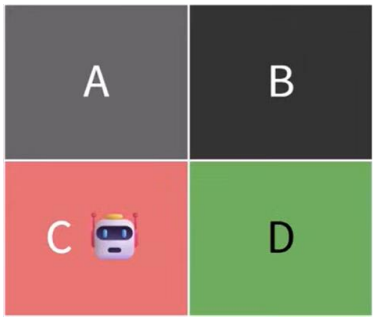
Trajectories





- Start: State A
- Action: East -> Reward: -1 New State: B





- Start: State A
- Action: South -> Reward: -10 New State: C

© 2020, UNIVERSITY OF MARYLAND

Markov Process

- A Markov process is a memoryless random process, i.e., a sequence of random states $S_1, S_2, \ldots$ with the Markov property.
- A Markov Process (or Markov Chain) is a tuple $\langle S, P \rangle$

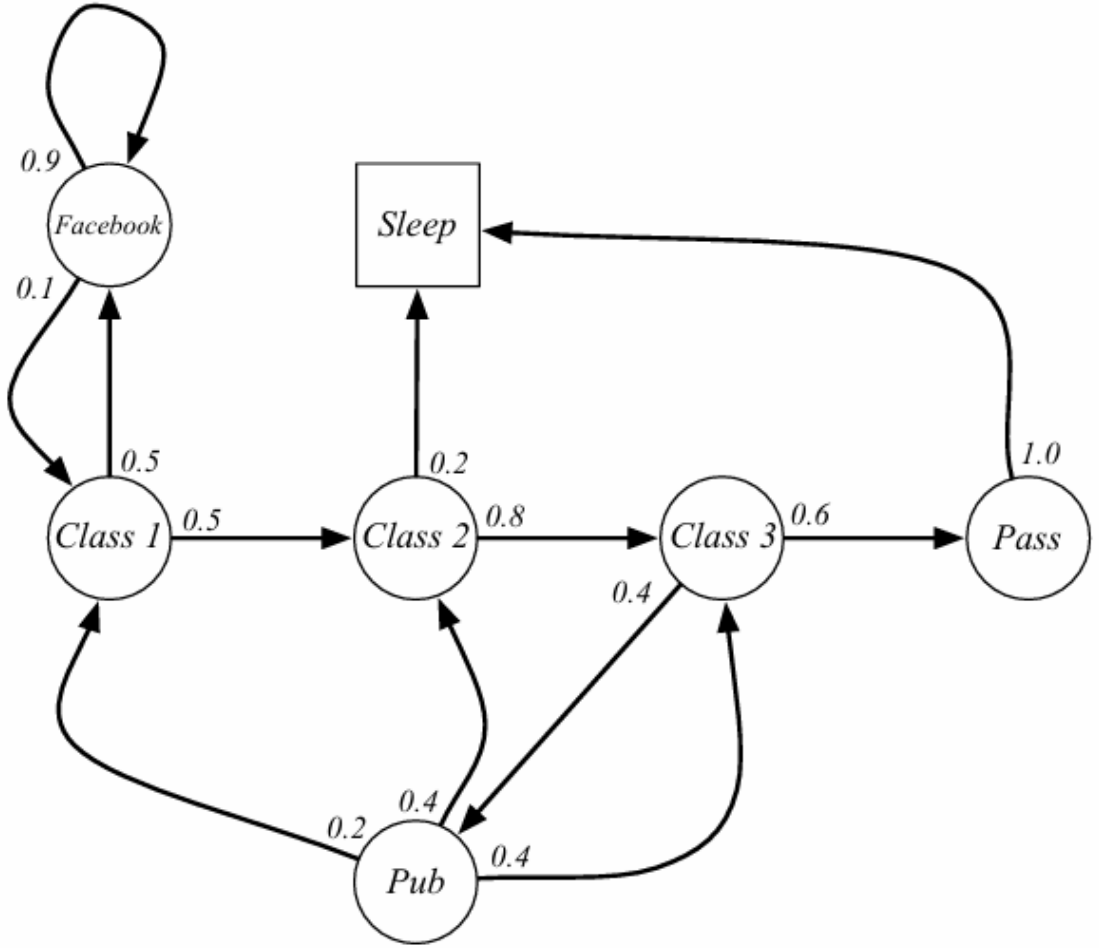
S is a **finite** set of states

P is a state transition probability matrix,

$$P_{ss'} = \mathbb{P}[S_{t+1} = s' | S_t = s]$$

© 2020, UNIVERSITY OF MARYLAND

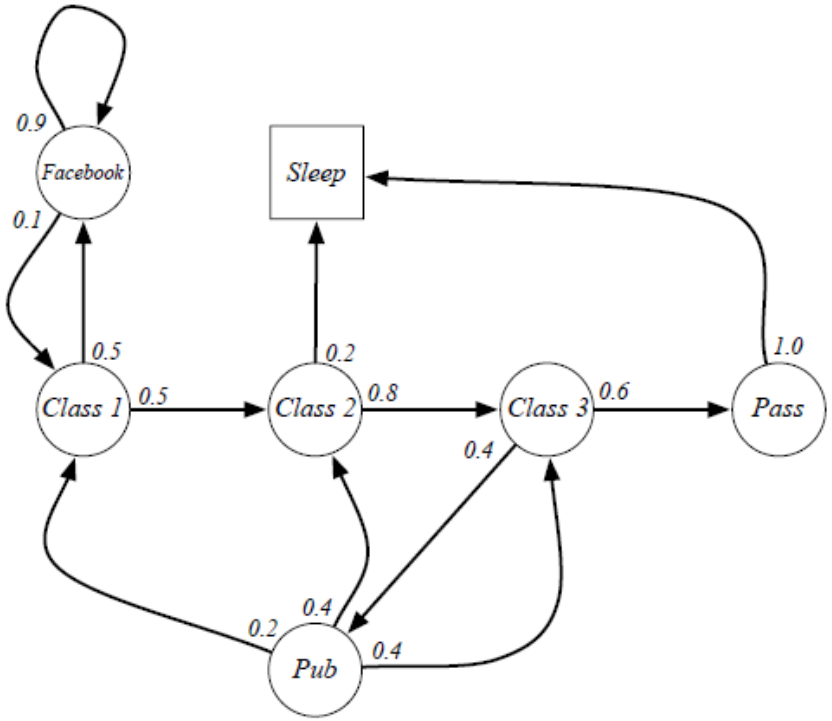
Example: Student Markov Chain



© 2020, UNIVERSITY OF MARYLAND

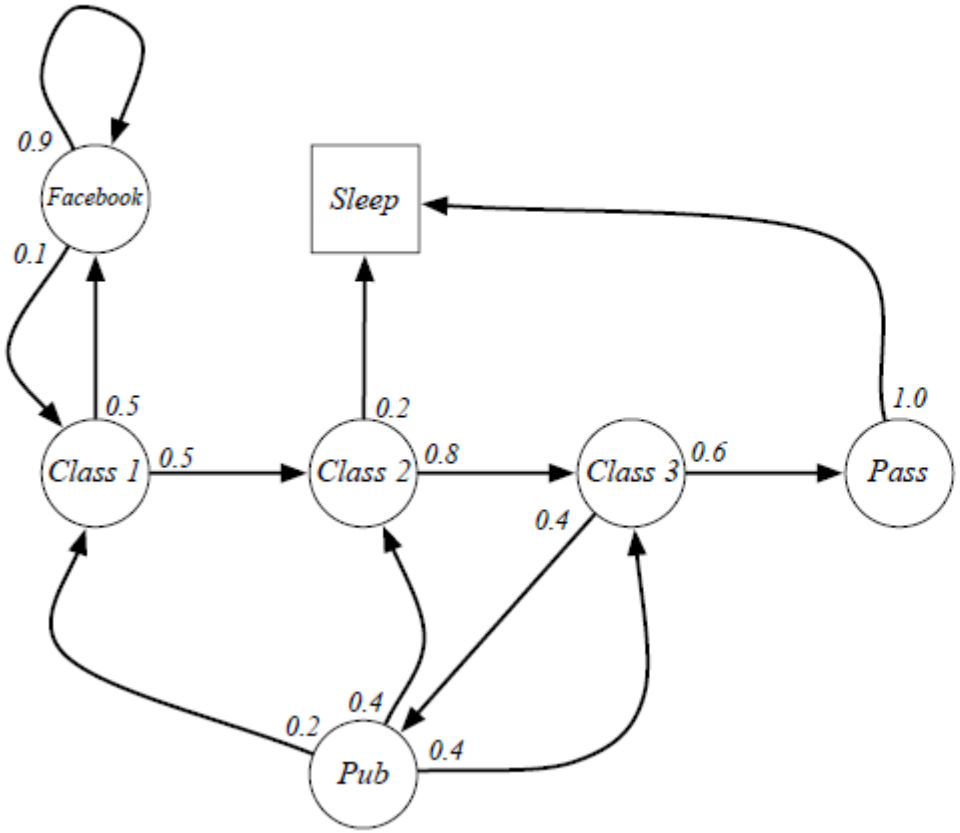
Example: Student Markov Chain Episodes

- Sample episodes for Student Markov Chain starting from $S_1 = C_1$
  - $S_1\ S_2\ S_3, \ldots S_T$
  - $C_1\ C_2\ C_3\ Pass\ Sleep$
  - $C_1\ FB\ FB\ C_1\ C_2\ Sleep$
  - $C_1\ C_2\ C_3\ Pub\ C_2\ C_3\ Pass\ Sleep$
  - $C_1\ FB\ FB\ C_1\ C_2\ C_3\ Pub\ C_1\ FB\ FB\ FB\ C_1\ C_2\ C_3\ Pub\ C_2\ Sleep$



© 2020, UNIVERSITY OF MARYLAND

Example: Student Markov Chain Transitions



$$\mathcal{P} = \begin{array}{c} \\ C1 \\ C2 \\ C3 \\ Pass \\ Pub \\ FB \\ Sleep \end{array} \begin{array}{c} \begin{array}{ccccccc} C1 & C2 & C3 & Pass & Pub & FB & Sleep \end{array} \\ \begin{bmatrix} & 0.5 & & & & 0.5 & \\ & & 0.8 & & & & 0.2 \\ & & & 0.6 & 0.4 & & \\ & & & & & & 1.0 \\ 0.2 & 0.4 & 0.4 & & & & \\ 0.1 & & & & & 0.9 & \\ & & & & & & 1 \end{bmatrix} \end{array}$$

© 2020, UNIVERSITY OF MARYLAND

Today

- We will assume that the MDP model is known
  - The distribution over outcomes (reward and next state) for each state and action
- Not a practical assumption but it will help us get started
- Assuming the same for the bandits problem directly implies the optimal policy
- Why is this not the case for MDPs?

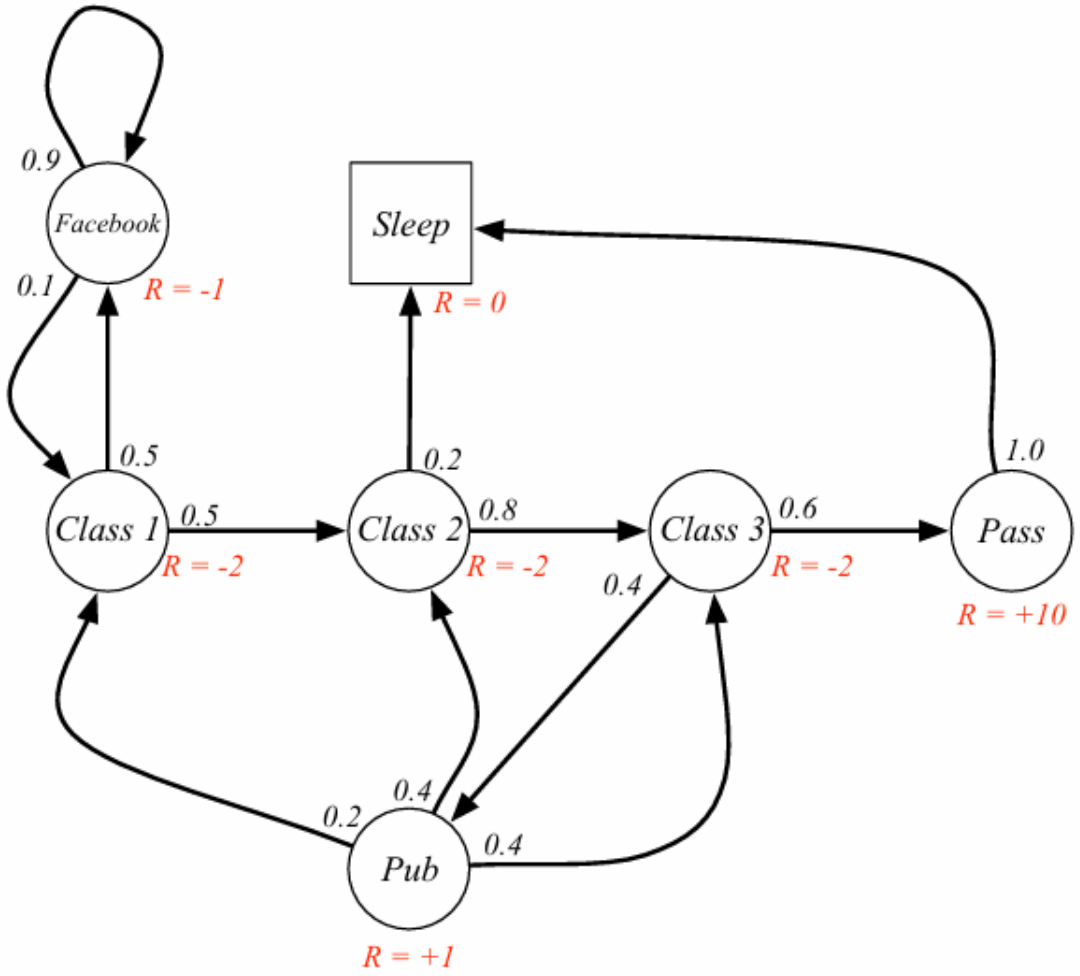
© 2020, UNIVERSITY OF MARYLAND

Markov Reward Process

- A Markov reward process is a Markov chain with values
- A Markov Reward Process is a tuple $\langle S, P, R, \gamma \rangle$
  - $\mathcal{S}$ is a finite set of states
  - P is a state transition probability matrix,
  - R is a reward function, $R_s = \mathbb{E}[R_{t+1} \mid S_t = s]$
  - γ is a discount factor, $0 \leq \gamma < 1$

© 2020, UNIVERSITY OF MARYLAND

Example: Student MRP



© 2020, UNIVERSITY OF MARYLAND

Return

- The return G_t is the total discounted reward from time-step t.

$$G_t = R_{t+1} + \gamma R_{t+2} + \ ... = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

- The discount $\gamma \epsilon [0, 1]$ is the present value of future rewards
- The value of receiving reward R after $k + 1$ time steps is $\gamma^k R$.
  - γ = 0: completely myopic; only immediate reward matters
  - γ close to 1: strongly values future rewards (far-sighted)
  - γ controls the agent's effective planning horizon

© 2020, UNIVERSITY OF MARYLAND

Discounting Solution

- Future rewards are assigned less weight than immediate rewards
- This creates a finite, comparable objective
- Uncertainty about the future may not be fully represented
- If the reward is financial, immediate rewards may earn more interest than delayed rewards
- Animal/human behavior shows preference for immediate reward

© 2020, UNIVERSITY OF MARYLAND

Discount Factor

- As the agent traverse the world it receives a sequence of rewards
- Which sequence has higher utility?
  - $\tau_1 = +1, +1, +1, +1, +1, +1\ldots$
  - $\tau_2 = +2, +2, +2, +2, +2, +2\ldots$
- Let's decay future rewards exponentially by a factor, $0 \leq \gamma < 1$

$$\sum_{t=0}^{\infty} \gamma^t r = \frac{r}{1-\gamma}$$ Geometric series

- Now τ_2 yields higher utility than τ_1

© 2020, UNIVERSITY OF MARYLAND

Discounting

- It's reasonable to maximize the sum of rewards
- It's also reasonable to prefer rewards now to rewards later
- Discount factor: values of rewards decay exponentially



1

Worth Now



γ

Worth Next Step



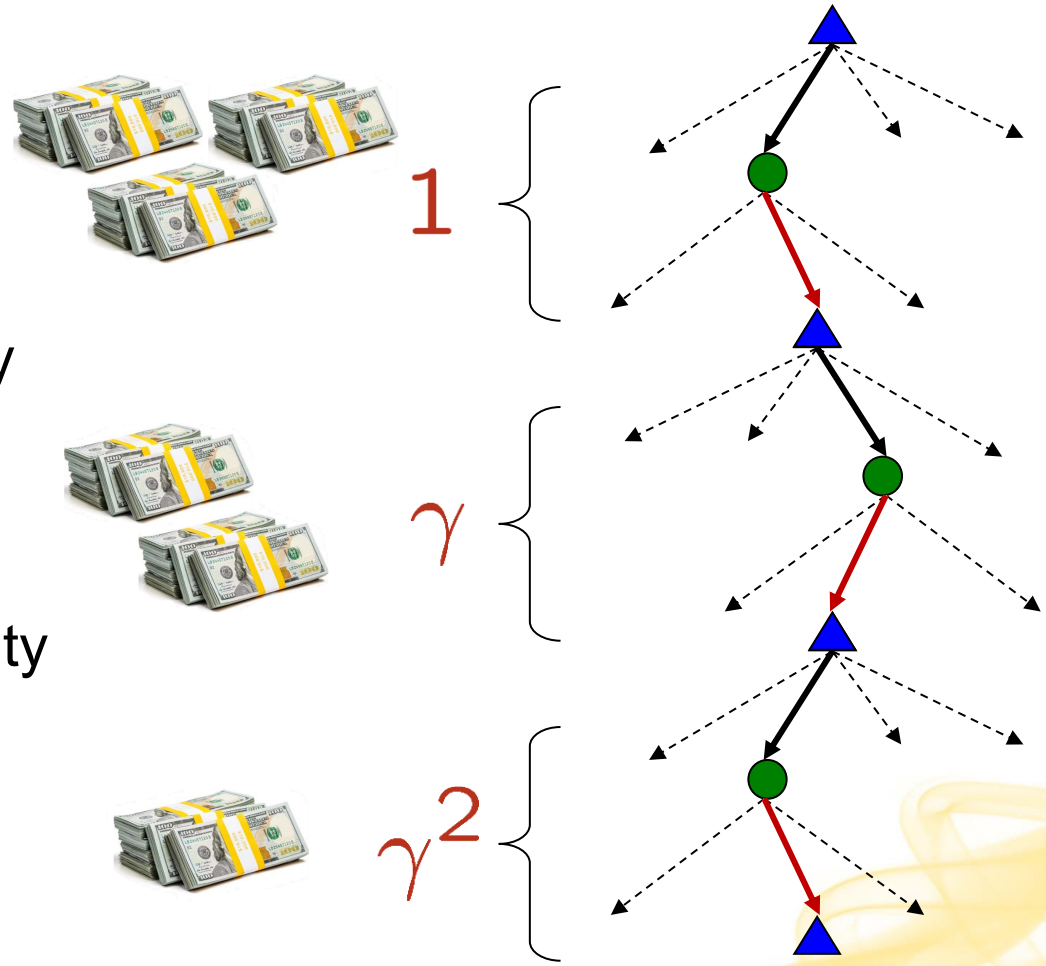
γ^2

Worth In Two Steps

© 2020, UNIVERSITY OF MARYLAND

Discounting

- How to discount?
  - Each time we descend a level, we multiply by the discount once
- Why discount?
  - Sooner rewards probably do have higher utility than later rewards
  - Also helps our algorithms converge
- Example: discount of 0.5
  - G(r=[1,2,3]) = 1*1 + 0.5*2 + 0.25*3
  - G([1,2,3]) < G([3,2,1])



26

26

© 2020, UNIVERSITY OF MARYLAND

Example: Continuous Task

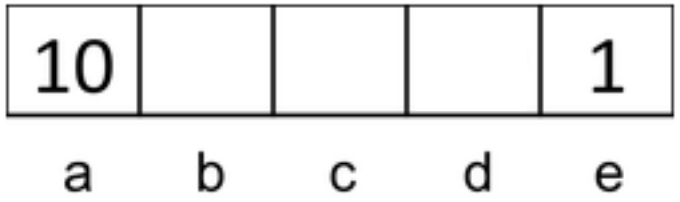
- Consider a non-episodic (continuous) task
- For a +1 reward forever and $\gamma = 0.9$: $G = 1 + 0.9 + 0.9^2 + \ldots$
- This is a geometric series that sums to 10
- An infinite stream becomes a finite optimization target

$$G_t = \frac{1}{1-\gamma} = \frac{1}{1-0.9} = \frac{1}{0.1} = 10$$

© 2020, UNIVERSITY OF MARYLAND

In-class Exercise: Discounting

- Given grid world:
  - Actions: East, West, and Exit ('Exit' only available in terminal states: a, e)
  - Rewards are given only after an exit action
  - Transitions: deterministic



- Q1: For $\gamma = 1$, what is the optimal policy?
- Q2: For $\gamma = 0.1$, what is the optimal policy?
- Q3: For which γ are West and East equally good when in state d?

© 2020, UNIVERSITY OF MARYLAND

Value Functions: The “Crystal Ball”

- A value function assigns a long-term score to a state
- It summarizes expected future return
- It lets the agent compare situations quantitatively

© 2020, UNIVERSITY OF MARYLAND

State Value Function

- The state value function $v(s)$ of MRP is the expected return starting from state s and following policy π

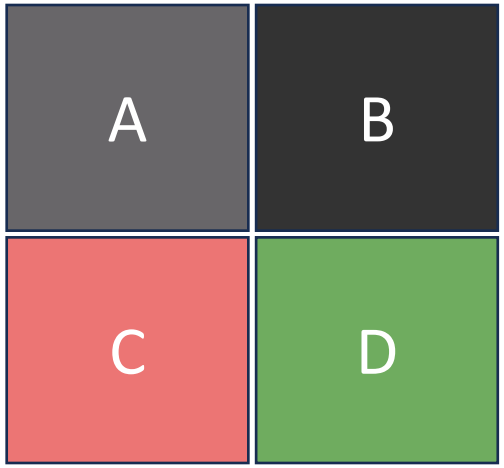
$$v_\pi(s) = \mathbb{E}[G_t \mid S_t = s]$$

- It answers, “How good is this state under this policy?”
- The value depends on both the environment and the policy

© 2020, UNIVERSITY OF MARYLAND

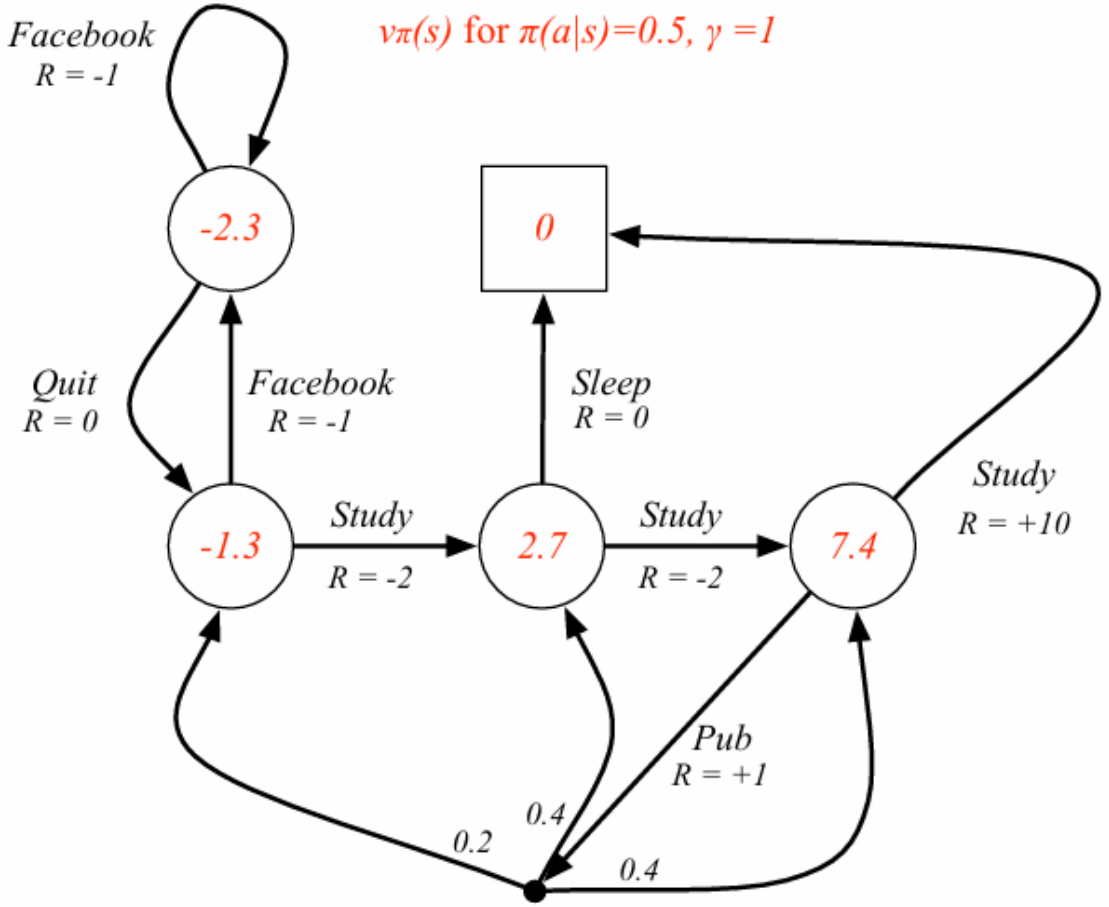
Example: State Value

- Under the policy, B is valuable because the goal is one move away
- A is slightly less valuable because reaching B costs −1
- The values quantify this intuition



© 2020, UNIVERSITY OF MARYLAND

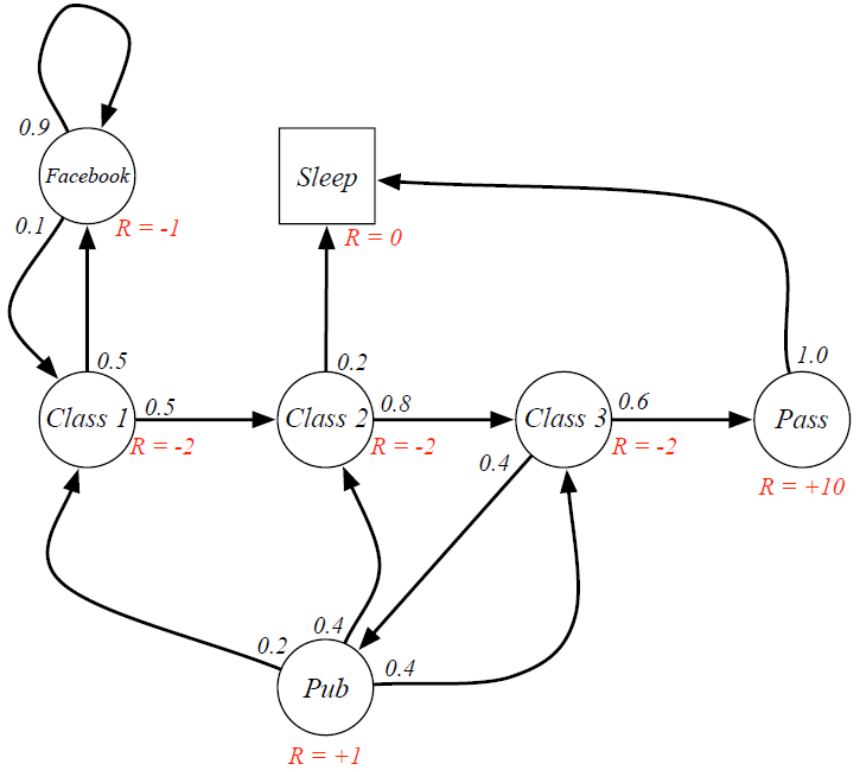
Example: State Value for Student MDP



© 2020, UNIVERSITY OF MARYLAND

Example: Student MRP

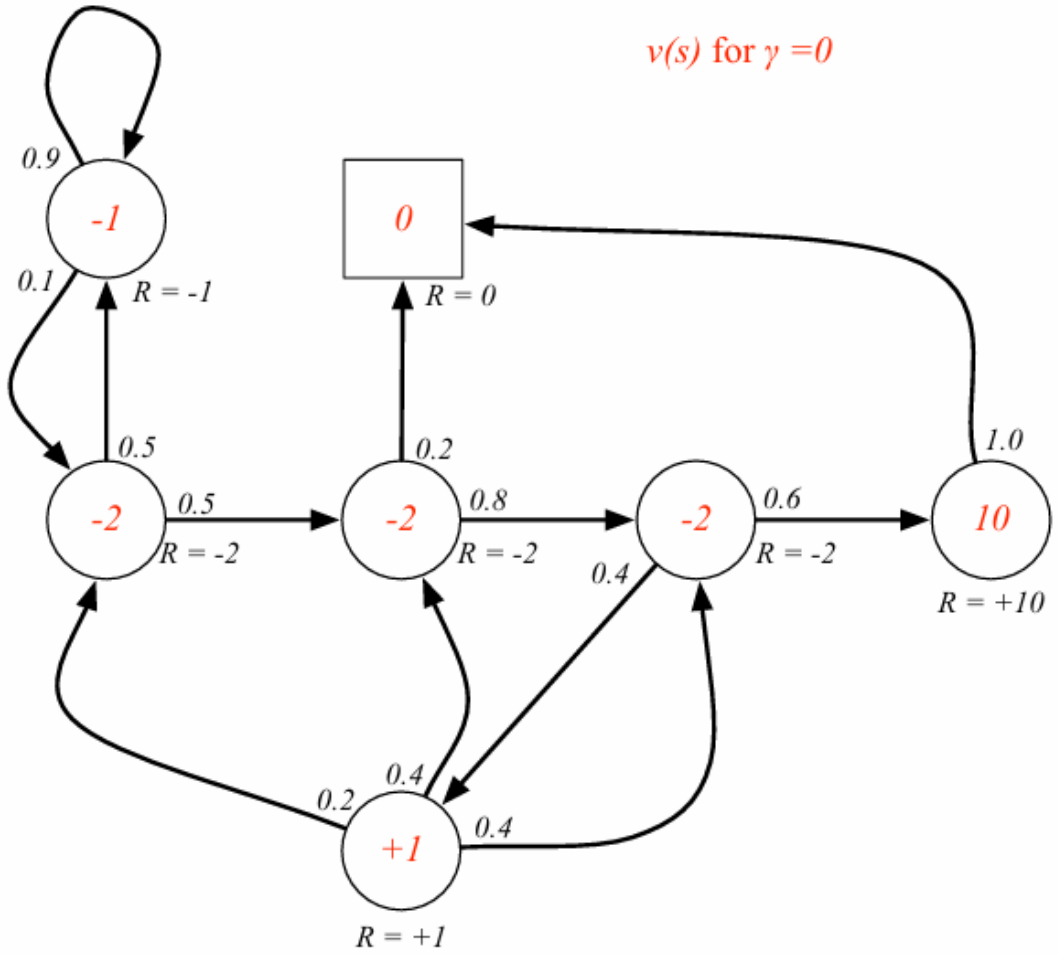
- Sample returns for Student MRP
- Start from $S_1 = C_1$ with $\gamma = 1/2$



C1 C2 C3 Pass Sleep	$v_1 = -2 - 2 * \frac{1}{2} - 2 * \frac{1}{4} + 10 * \frac{1}{8}$	=	-2.25
C1 FB FB C1 C2 Sleep	$v_1 = -2 - 1 * \frac{1}{2} - 1 * \frac{1}{4} - 2 * \frac{1}{8} - 2 * \frac{1}{16}$	=	-3.125
C1 C2 C3 Pub C2 C3 Pass Sleep	$v_1 = -2 - 2 * \frac{1}{2} - 2 * \frac{1}{4} + 1 * \frac{1}{8} - 2 * \frac{1}{16} \ldots$	=	-3.41
C1 FB FB C1 C2 C3 Pub C1 ...	$v_1 = -2 - 1 * \frac{1}{2} - 1 * \frac{1}{4} - 2 * \frac{1}{8} - 2 * \frac{1}{16} \ldots$	=	-3.20
FB FB FB C1 C2 C3 Pub C2 Sleep			

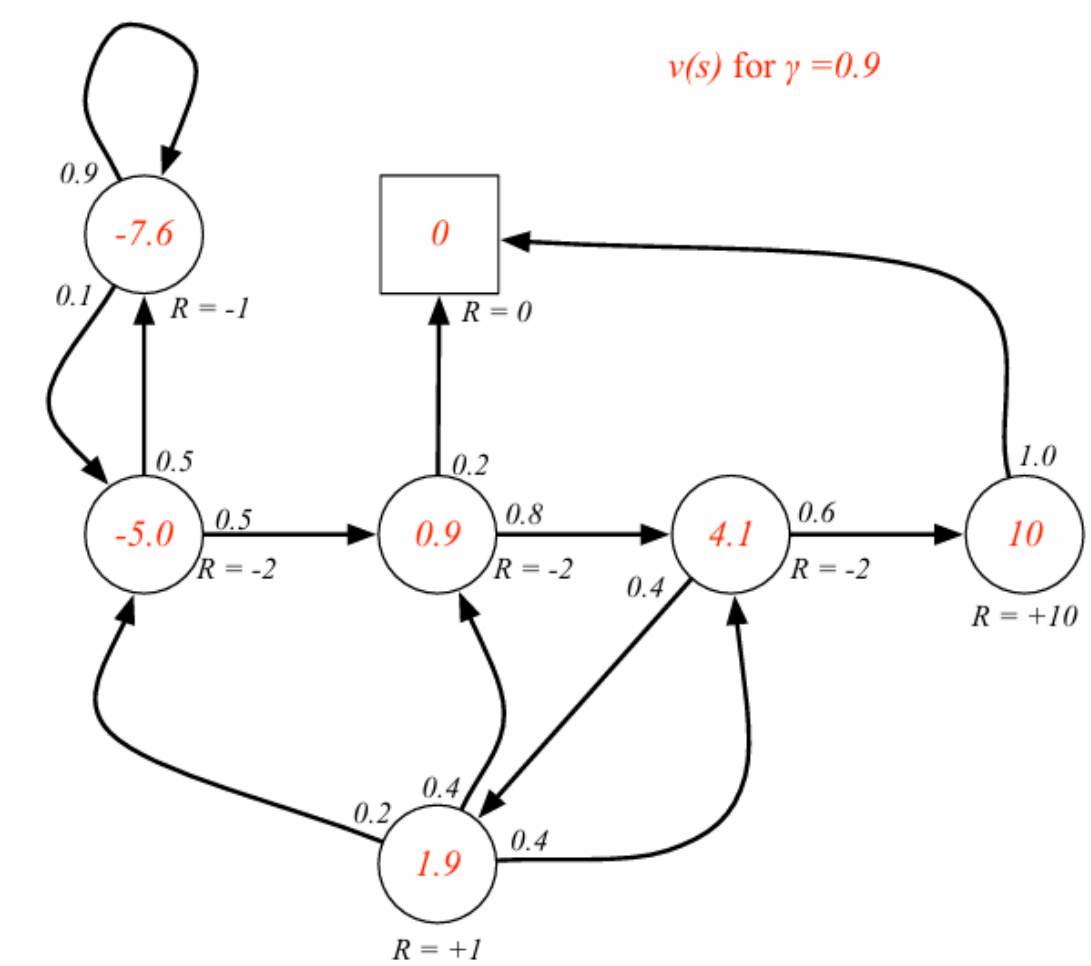
© 2020, UNIVERSITY OF MARYLAND

Example: State-Value for Student MRP



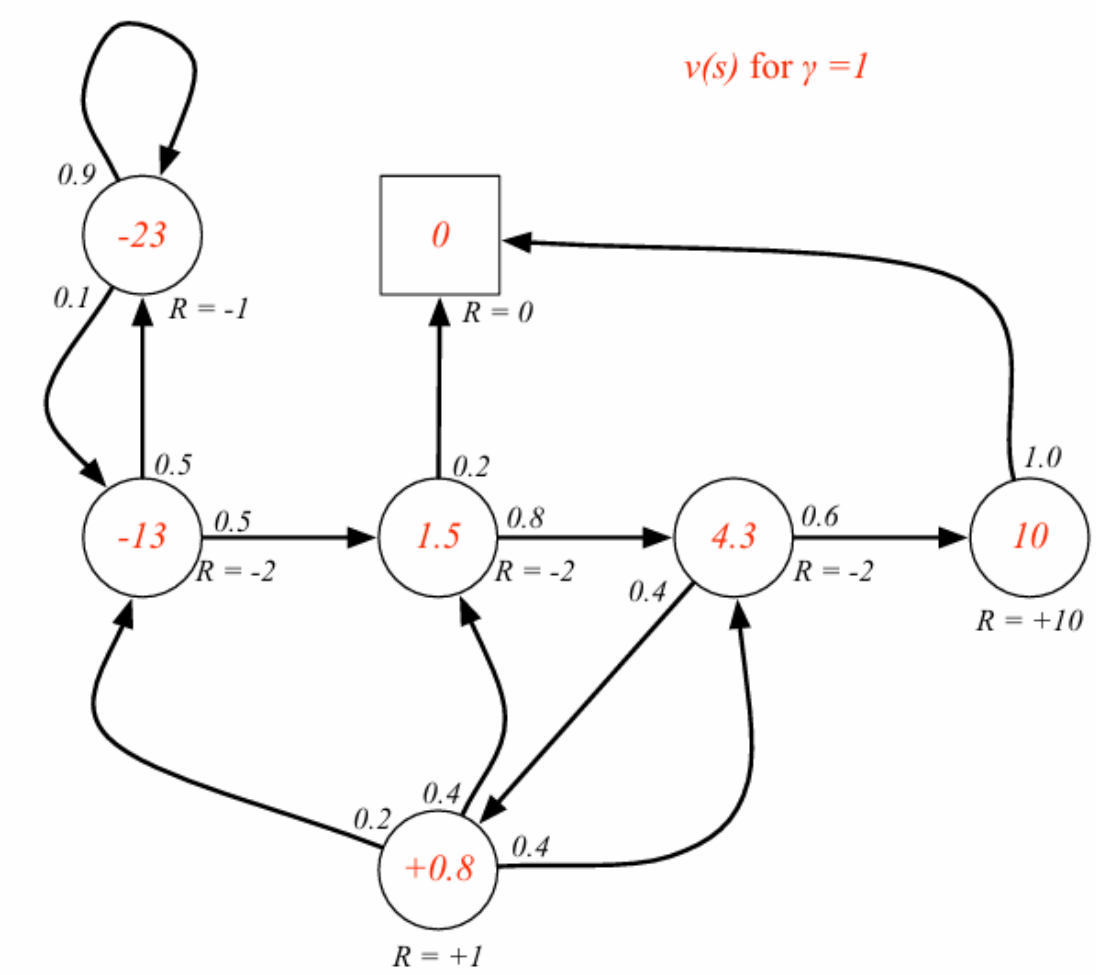
© 2020, UNIVERSITY OF MARYLAND

Example: State-Value for Student MRP



© 2020, UNIVERSITY OF MARYLAND

Example: State-Value for Student MRP



© 2020, UNIVERSITY OF MARYLAND

Bellman Equation for MRPs

- The value function can be decomposed into two parts:
  - Immediate reward R_{t+1}
  - Discounted value of successor state $\gamma v(S_{t+1})$

$$
\begin{aligned}
v_\pi(s) &= \mathbb{E}[G_t \mid S_t = s] \\
&= \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots \mid S_t = s] \\
&\ldots \\
&= \mathbb{E}[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s]
\end{aligned}
$$

© 2020, UNIVERSITY OF MARYLAND

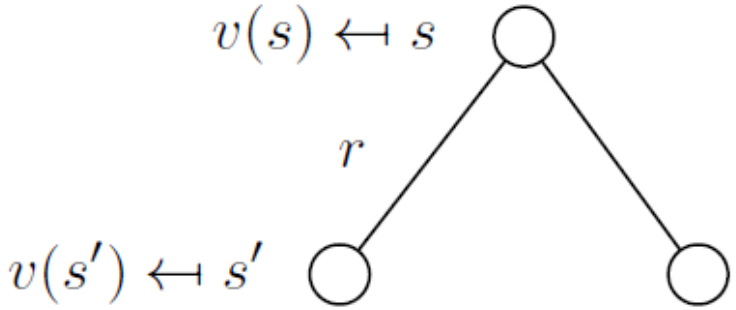
Bellman Equation: Intuition

- Current value = immediate reward + discounted value of the next state
- A long-term problem can therefore be expressed using one-step relationships
- This recursive structure is the Bellman equation
- Future value 'echoes back' to determine present value
- Neighboring states become mathematically linked
- The policy determines how actions are weighted
- Transition dynamics determine possible next states and rewards

© 2020, UNIVERSITY OF MARYLAND

Bellman Equation for MRPs

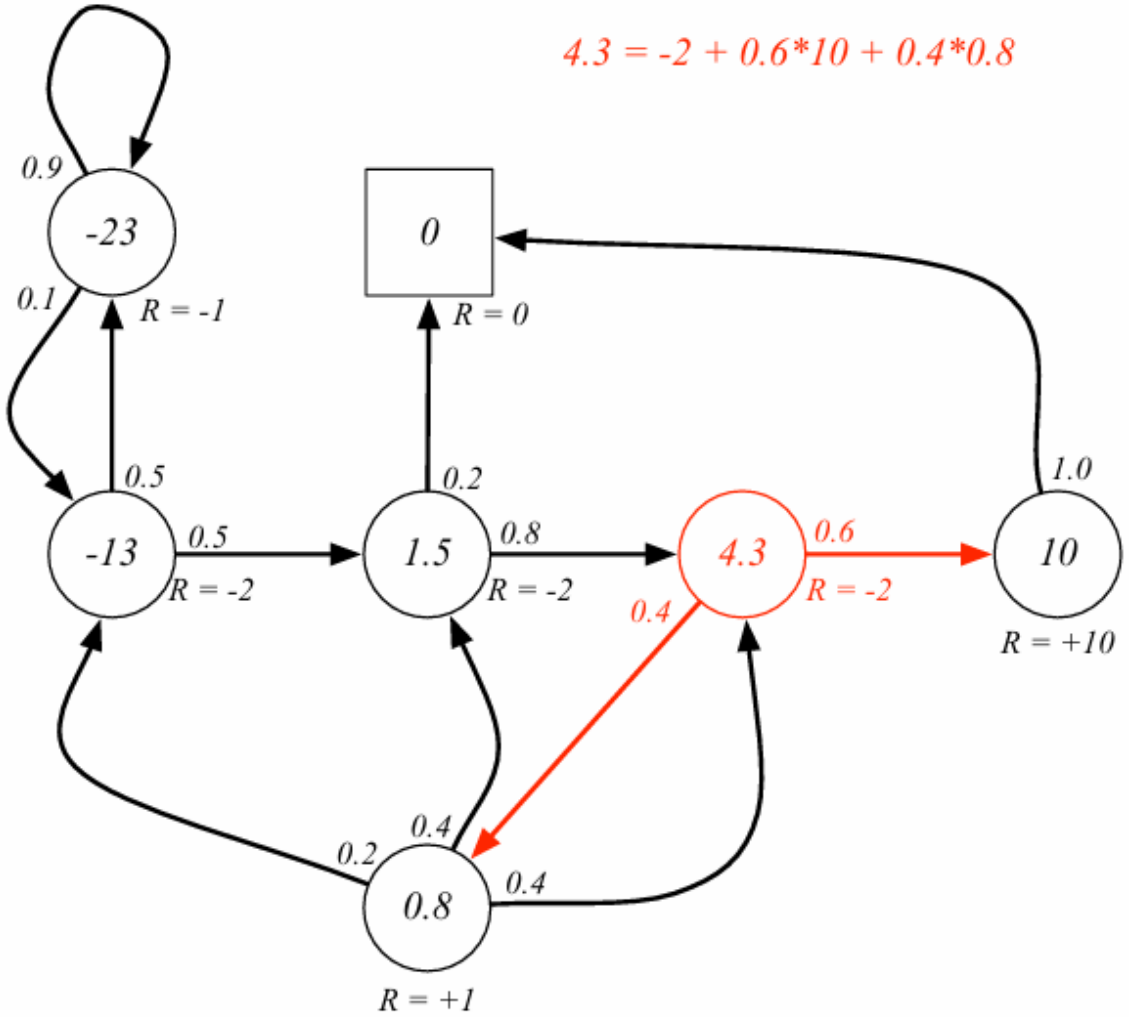
$$v(s) = \mathbb{E}[R_{t+1} + \gamma v(S_{t+1} \mid S_t = s]$$



$$v(s) = \mathcal{R}_s + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'} v(s')$$

© 2020, UNIVERSITY OF MARYLAND

Example: Bellman Equation for Student MRP



© 2020, UNIVERSITY OF MARYLAND

Bellman Equation in Matrix Form

- The Bellman equation can be expressed concisely using matrices

$$v = R + \gamma P\ v$$

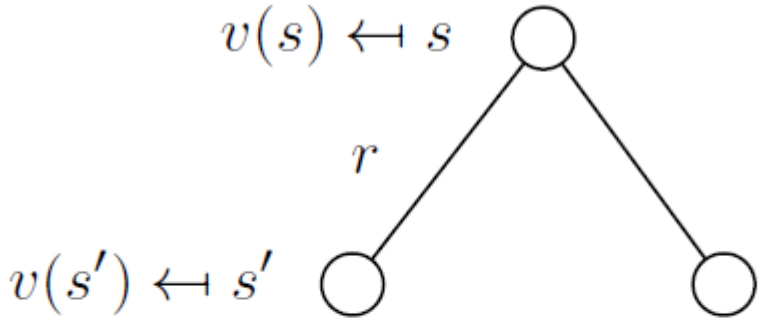
where v is a column vector with one entry per state

$$\begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix} = \begin{bmatrix} \mathcal{R}_1 \\ \vdots \\ \mathcal{R}_n \end{bmatrix} + \gamma \begin{bmatrix} \mathcal{P}_{11} & \dots & \mathcal{P}_{1n} \\ \vdots & & \\ \mathcal{P}_{11} & \dots & \mathcal{P}_{nn} \end{bmatrix} \begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix}$$

© 2020, UNIVERSITY OF MARYLAND

Bellman Equation in Matrix Form

$$v_\pi(s) = \mathbb{E}[R_{t+1} + \gamma v(S_{t+1} \mid S_t = s]$$



$$v(s) = \mathcal{R}_s + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'} v(s')$$

© 2020, UNIVERSITY OF MARYLAND

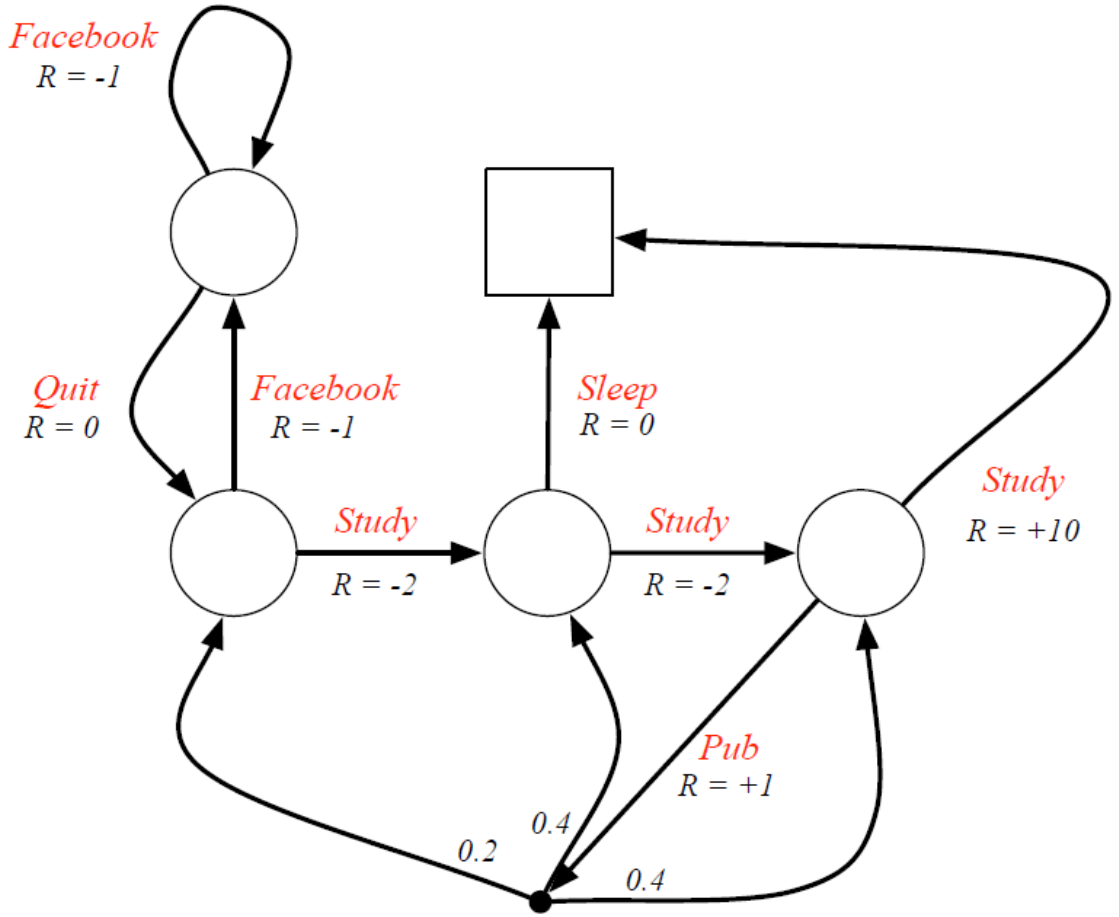
Formally, MDP

- An MDP is a memoryless random process, defined by:
  - A set of (finite) states $\mathcal{S}$
  - A set of actions $\mathcal{A}$
  - State-transition probabilities $P(s'|s,a)$
    - Probability of arriving to s' after performing a at s
    - Also called the model dynamics
  - A reward function $R(s,a,s')$
    - The utility gained from arriving at s' after performing a at s
    - Sometimes just $R(s,a)$ or even $R(s)$
  - A start state
  - Maybe a terminal state

$$\mathcal{P} = \text{from} \begin{bmatrix} & \text{to} & \\ \mathcal{P}_{11} & \cdots & \mathcal{P}_{1n} \\ \vdots & & \\ \mathcal{P}_{n1} & \cdots & \mathcal{P}_{nn} \end{bmatrix}$$

© 2020, UNIVERSITY OF MARYLAND

Example: Student MDP



© 2020, UNIVERSITY OF MARYLAND

MDP formalization - Autonomous driving

- State:
  - image from front camera
- Actions:
  - Steering from [-1,1], acceleration from [0,1], brake from [0,1]
- Reward:
  - Drive off road = -10
  - Hit a person = -1000
  - Drive at speed limit = +2
  - Run a stop sign = -5
- State-transition probabilities:
  - defined by stochasticity in action outcomes and other actuators

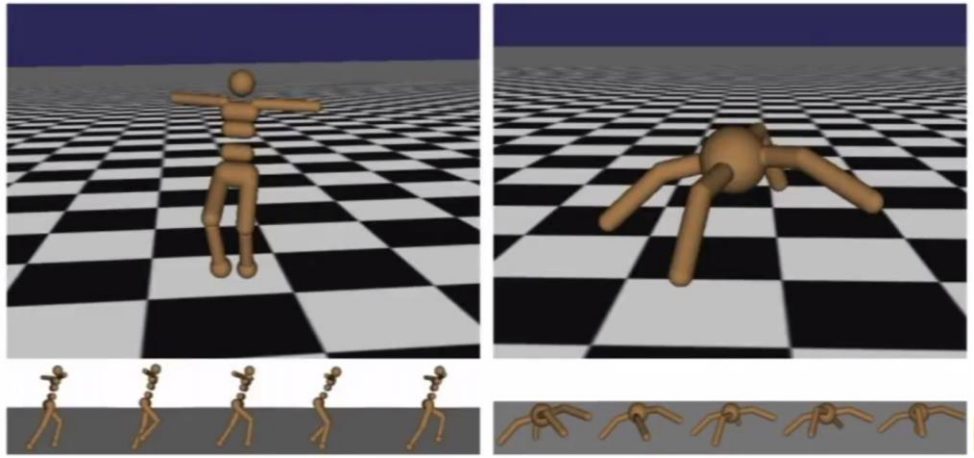


["Learning to Drive in a Day", Kendall et. al., 2018]

© 2020, UNIVERSITY OF MARYLAND

MDP formalization – Robot locomotion

- State:
  - angle and position of joints, obstacles/pits
- Actions:
  - Torques applied to joints
- Reward:
  - Forward speed
- State-transition probabilities:
  - defined by stochasticity in action outcomes and obstacles



["Policy gradient reinforcement learning for fast quadrupedal locomotion", Kohl & Stone, 2004]

© 2020, UNIVERSITY OF MARYLAND

Feedback Control For Cassie With Deep Reinforcement Learning

Zhaoming Xie[1], Glen Berseth[1], Patrick Clary[2], Jonathan Hurst[2], Michiel van de Panne[1]

[1]University of British Columbia, [2]Oregon State University

Policy

- Policy π is the agent's behavior; a distribution over actions given states
- It is a map from state to action
- MDP policies depend on the current state (not the history)
- Policies are stationary (time-independent)
- Deterministic policy

$$\mathrm{a} = \pi(s), \pi(a|s) = 1$$

- Stochastic policy

$$\mathrm{a} = \pi(a|s) = P[A_t = a \mid S_t = s]$$

© 2020, UNIVERSITY OF MARYLAND

Policy

- Given an MDP y $M = \langle S, A, P, R, \gamma \rangle$ and policy π
- The state sequence $S_1, S_2, \ldots$ is a Markov process $\langle S, P^{\pi} \rangle$
- The state and reward sequence $S_1, R_1, S_2, \ldots$ is a Markov reward process $\langle S, P^{\pi}, R^{\pi}, \gamma \rangle$ where

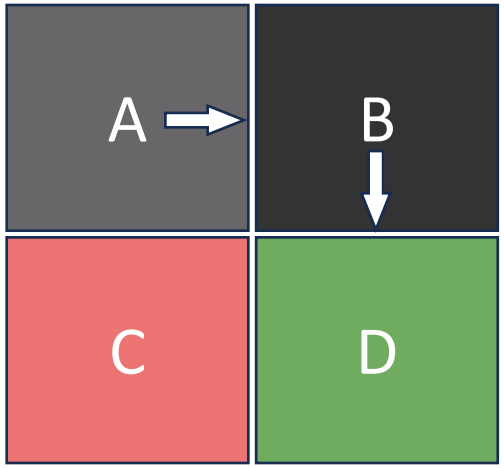
$$\mathcal{P}^{\pi}_{s,s'} = \sum_{a \in \mathcal{A}} \pi(a|s) \mathcal{P}^{a}_{ss'}$$

$$\mathcal{R}^{\pi}_{s} = \sum_{a \in \mathcal{A}} \pi(a|s) \mathcal{R}^{a}_{s}$$

© 2020, UNIVERSITY OF MARYLAND

Example Policy #1

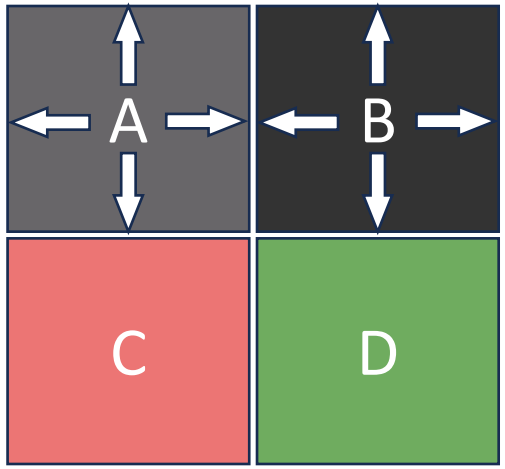
- Rule for A: *always* go East
- Rule for B: *always* go South
- This is a deterministic policy. No randomness
- It chooses one action with probability 1



© 2020, UNIVERSITY OF MARYLAND

Example Policy #2

- Rule for ANY state: Choose N, S, E, W, randomly
- This is a stochastic policy – assigns probabilities to actions
- Choose north, south, east, and west with probability 0.25 each

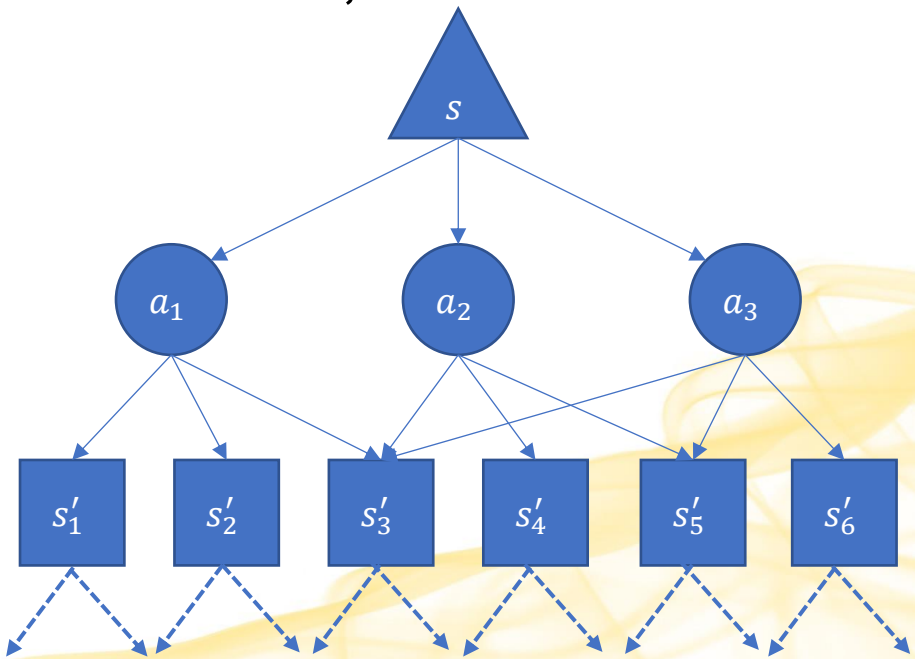


© 2020, UNIVERSITY OF MARYLAND

MDP Objective

A set of states $s \in \mathcal{S}$
A set of actions $a \in \mathcal{A}$
State-transition probabilities $P(s'|s,a)$
A reward function $R(s,a,s')$

- Compute a policy: what action to take at each state
  - $\pi: S \rightarrow A$
- Compute the **optimal** policy: maximum expected reward, π^*
- $\pi^*(s) = ?$
- $= \underset{a}{\text{argmax}}[\sum_{s'} P(s'|s,a)R(s,a,s')]$
  - Must also optimize over the future (next steps)
- $= \underset{a}{\text{argmax}}[\sum_{s'} P(s'|s,a)\ (R(s,a,s') + \underbrace{\mathbb{E}_{\pi^*}[G|s']}_{v^*(s')})]$



© 2020, UNIVERSITY OF MARYLAND

Notation

- π^* - a policy that yields the maximum expected sum of rewards
- G- observed sum of rewards, i.e., $\sum r_t$ (return)
- $v^*(s)$ - the expected sum of rewards from being at s then following

$$\pi^* = \mathbb{E}_{\pi^*}\left[G|s\right]$$

© 2020, UNIVERSITY OF MARYLAND

Action Value Function

- The action value function $q_\pi(s, a)$ is the expected return starting from state s, taking action a and then following policy π

$$q_\pi(s, a) = \mathbb{E}[G_t \mid S_t = s, A = a]$$

- It evaluates a specific action from a state
- It asks: 'How good is this move, followed by policy π afterward?'
- Q-values are directly useful for selecting actions

© 2020, UNIVERSITY OF MARYLAND

V vs Q

- $v_\pi(s)$ How good is the state?
- $q_\pi(s, a)$ How good is a particular action from that state?
- V averages over actions according to the policy π
- Q distinguishes the alternatives
  - If Q -values are known, choose the action with the largest Q -value
  - This turns action selection into a comparison problem
  - Q -values become central to many reinforcement-learning algorithms

© 2020, UNIVERSITY OF MARYLAND

Bellman Expectation Equation

- The state-value function can be decomposed into immediate reward plus discounted value of successor state,

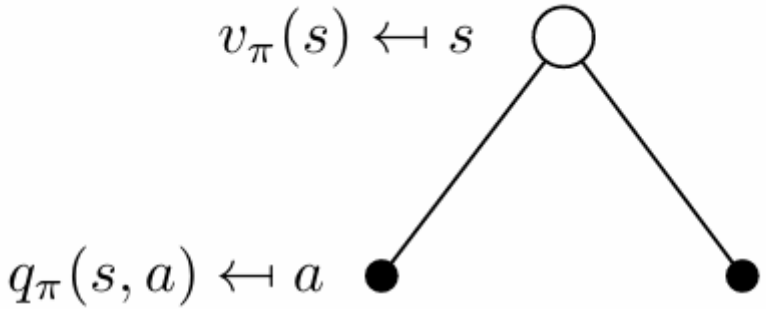
$$v_\pi(s) = \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]$$

- Similarly, the action-value function can be decomposed as follows,

$$q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A = a]$$

© 2020, UNIVERSITY OF MARYLAND

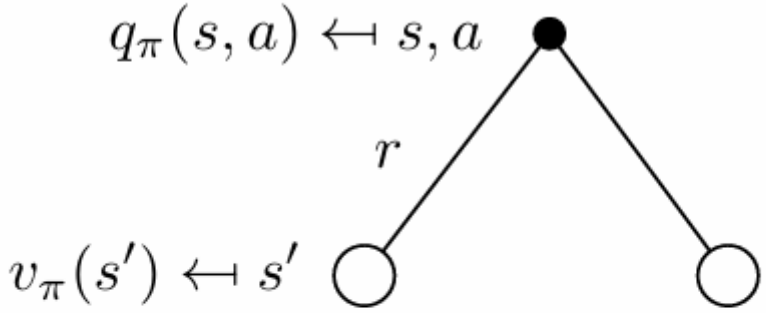
Bellman Expectation Equation for v_π



$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) q_\pi(s,a)$$

© 2020, UNIVERSITY OF MARYLAND

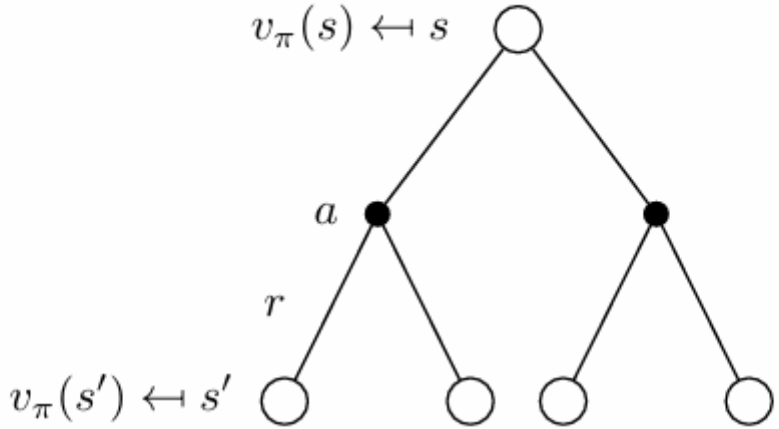
Bellman Expectation Equation for q_π



$$q_\pi(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s')$$

© 2020, UNIVERSITY OF MARYLAND

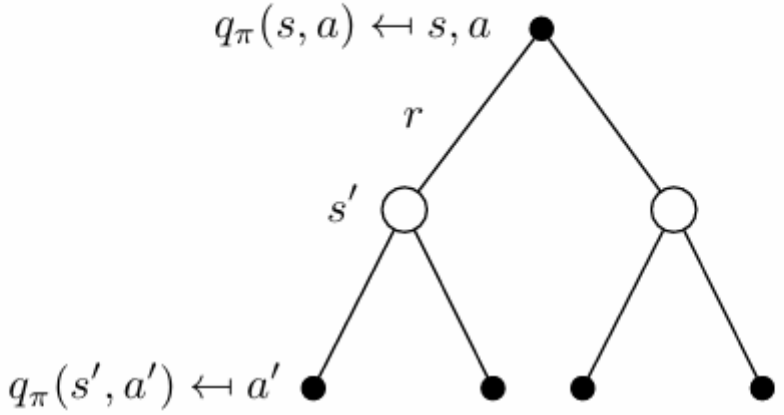
Bellman Expectation Equation for v_π



$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s') \right)$$

© 2020, UNIVERSITY OF MARYLAND

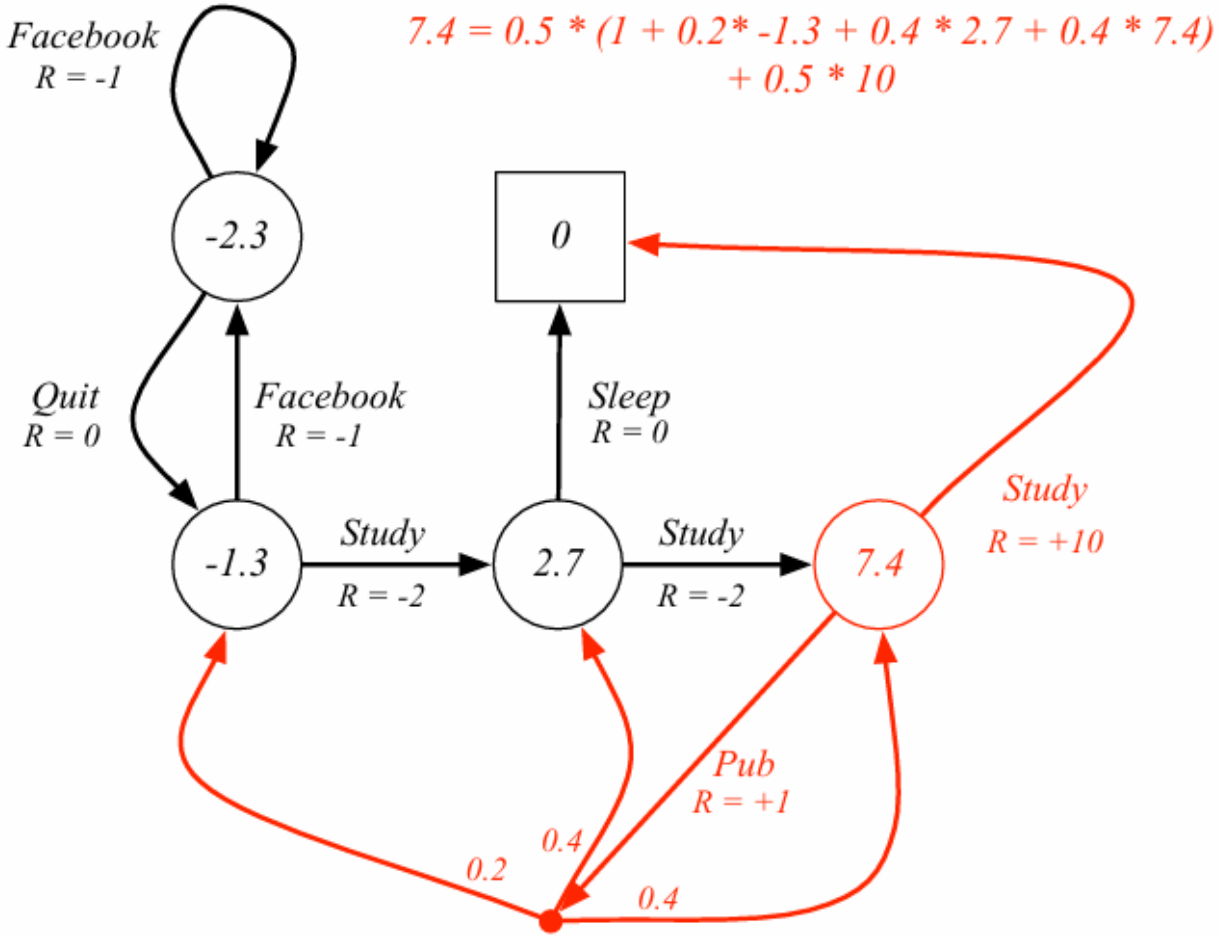
Bellman Expectation Equation for q_π



$$q_\pi(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a'|s') q_\pi(s',a')$$

© 2020, UNIVERSITY OF MARYLAND

Example: Bellman Expectation in Student MDP



© 2020, UNIVERSITY OF MARYLAND

Optimal Value Function

- The optimal state-value function $v^*(s)$ is the maximum value function over all policies

$$v^*(s) = \max_{\pi}[v_\pi(s)]$$

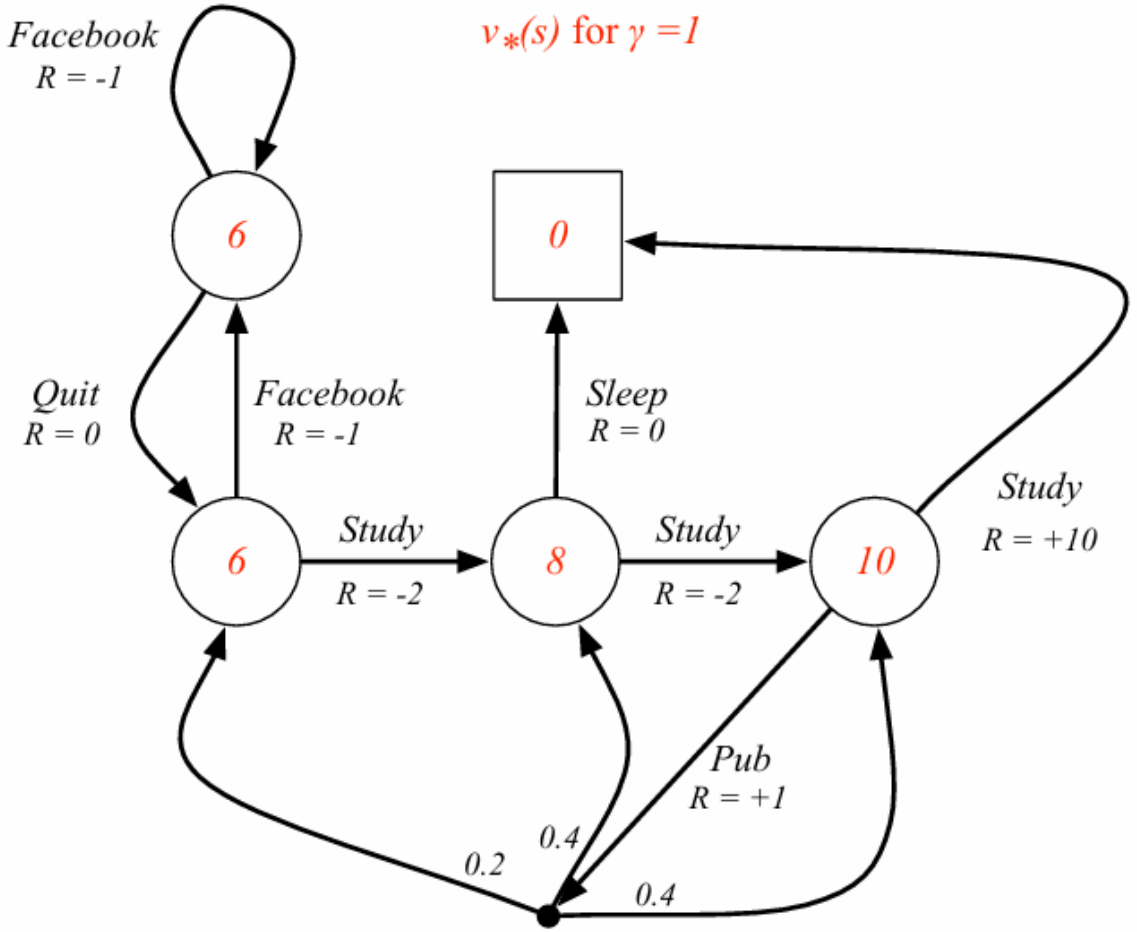
- The optimal action-value function $q^*(s,a)$ is the maximum action-value function over all policies

$$q^*(s,a) = \max_{\pi}[q_\pi(s,a)]$$

- The optimal value function species the best possible performance in MDP

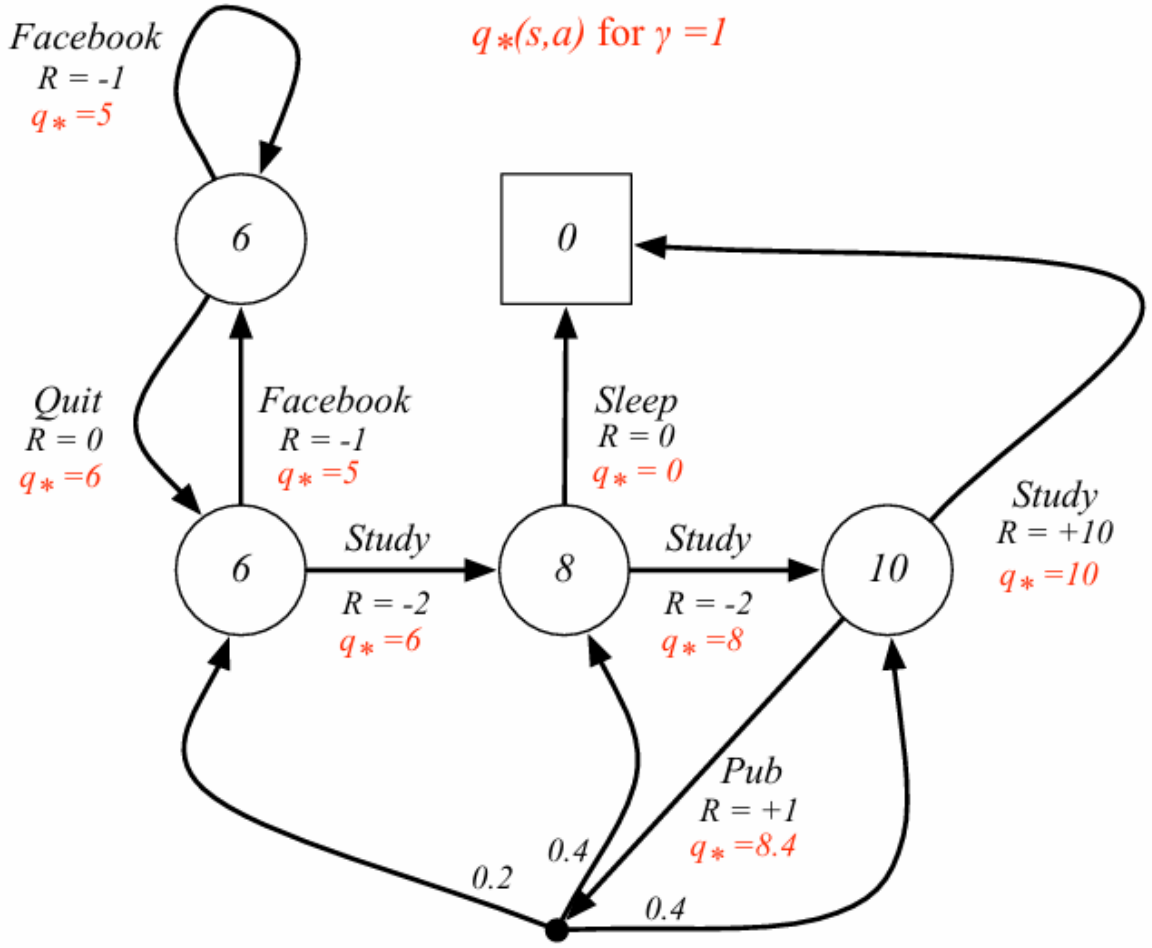
© 2020, UNIVERSITY OF MARYLAND

Optimal Value Function for Student MDP



© 2020, UNIVERSITY OF MARYLAND

Optimal Action Value Function for Student MDP



© 2020, UNIVERSITY OF MARYLAND

Optimal Policy

- For any Markov Decision Process
  - There exists an optimal policy π^* that is better than or equal to all other policies, $\pi^* \geq \pi, \forall \pi$
  - All optimal policies achieve the optimal value function, $v_\pi^*(s) = v^*(s)$
  - All optimal policies achieve the optimal action-value function, $q_\pi^*(s,a) = q^*(s,a)$

© 2020, UNIVERSITY OF MARYLAND

Finding Optimal Policy

- An optimal policy can be found by maximizing over $q^*(s, a)$

$$\pi_*(a|s) = \begin{cases} 1 & \text{if } a = \underset{a \in \mathcal{A}}{\text{argmax}}\ q_*(s, a) \\ 0 & \textit{otherwise} \end{cases}$$

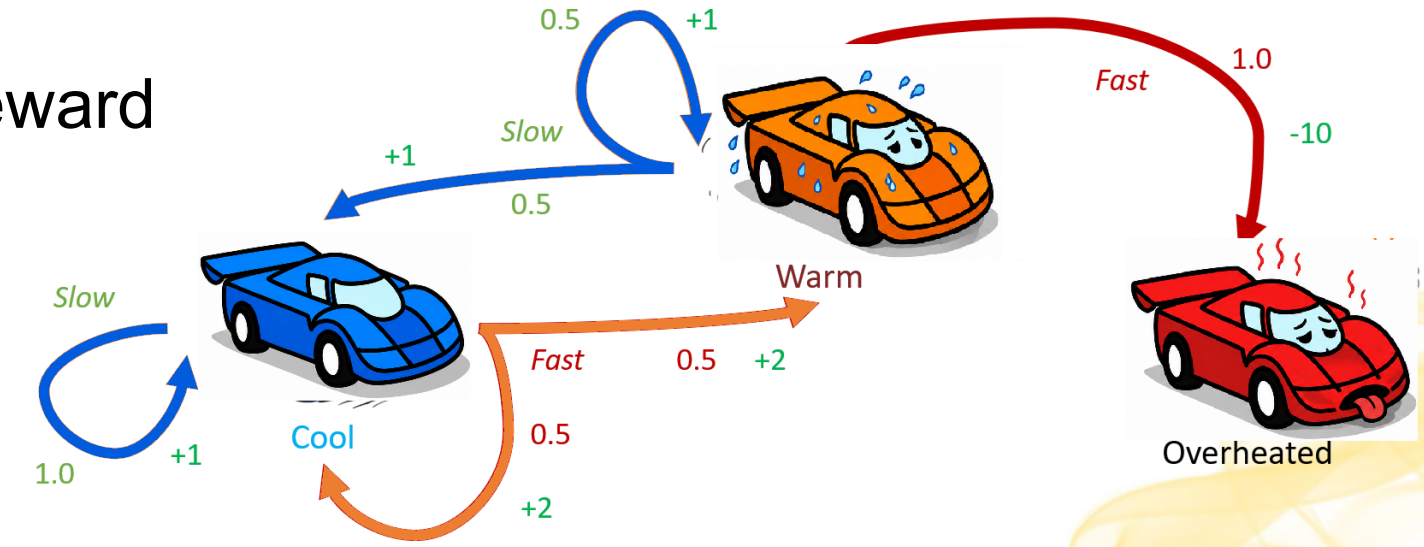
- There is always a deterministic optimal policy for any MDP
- If we know $q^*(s, a)$, we immediately have the optimal policy

© 2020, UNIVERSITY OF MARYLAND

Example: Race Car

- A robot car wants to travel far, quickly
- Three states: Cool, Warm, Overheated
- Two actions: *Slow*, *Fast*
- Going faster gets double reward
- What is $V^*(\text{Cool})$?
- What is $\pi^*(\text{Cool})$?

$$\max_a \left[\sum_{s'} P(s'|s,a) \left(R(s,a,s') + v^*(s') \right) \right]$$



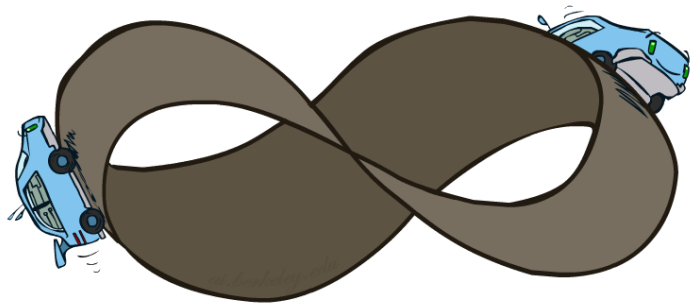
© 2020, UNIVERSITY OF MARYLAND

Infinite Utilities?!

- Problem: What if the game lasts forever? Do we get infinite rewards?
- Solutions:
  - Finite horizon: (similar to depth-limited search)
    - Terminate episodes after a fixed T steps (e.g. life)
    - Gives nonstationary policies (π depends on time left)
  - Discounting: use $0 < \gamma < 1$

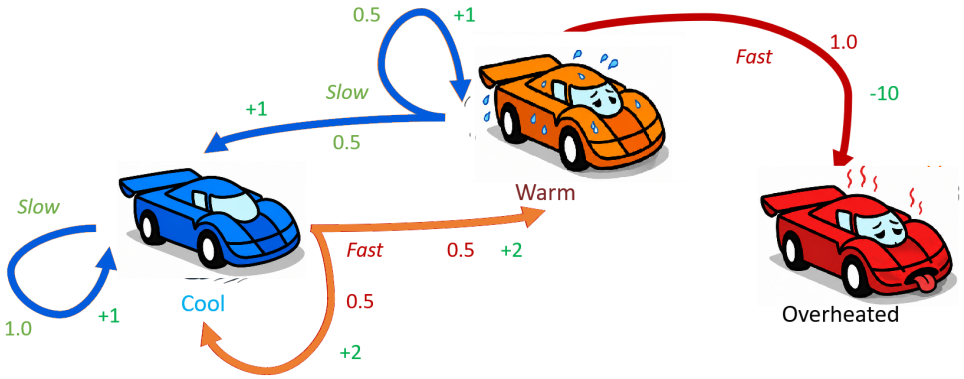
$$U([r_0, \ldots r_\infty]) = \sum_{t=0}^{\infty} \gamma^t r_t \leq R_{\text{max}}/(1-\gamma)$$

  - Smaller γ means smaller "horizon" – shorter term focus



© 2020, UNIVERSITY OF MARYLAND

Example: Race car



- Consider a discount factor, $\gamma = 0.9$
- What is $v^*(Cool)$

$$= \max_a \left[r(s,a) + \sum_{s'} p(s'|s,a) \ \gamma \ v^*(s') \right]$$

$$= \max[1 + 0.9 \cdot 1 v^*(Cool), 2 + 0.9 \cdot 0.5 v^*(Cool) + 0.9 \cdot 0.5 v^*(Warm)]$$

© 2020, UNIVERSITY OF MARYLAND

Solving the Bellman Equation

$$v = R + \gamma P\, v$$

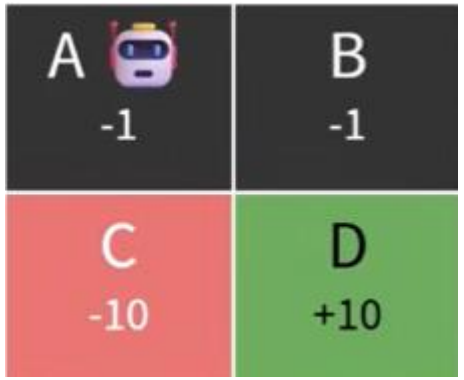
$$v = (1 - \gamma P)^{-1} R$$

- The Bellman equation is a linear equation
- It can be solved directly
- Computational complexity is $O(n^3)$ for n states
- Direct solution only possible for small MRPs
- There are many iterative methods for large MRPs, e.g.
  - Dynamic programming
  - Monte-Carlo evaluation
  - Temporal-Difference learning

© 2020, UNIVERSITY OF MARYLAND

Example: Simple Maze

1. Go North (hit wall, stay in A) = -1 + 0.9 * $v(A)$
2. Go West (hit wall, stay in A) = -1 + 0.9 * $v(A)$
3. Go East (move to B) = -1 + 0.9 * $v(B)$
4. Go South (move to C, pit) = -10 + 0.9 * $v(C)$ = -10



$$v(A) = 0.25[-1 + 0.9\ v(A)] + 0.25\ [-1 + 0.9\ v(A)] + 0.25[-1 + 0.9v(B)] + 0.25[-10]$$

$$v(B) = 0.25[-1 + 0.9\ v(B)] + 0.25\ [-1 + 0.9\ v(B)] + 0.25[-1 + 0.9v(A)] + 0.25[+10]$$

CIRCULAR DEPENDENCY!

For a small state space, we can use system of equations?

© 2020, UNIVERSITY OF MARYLAND

Direct Solution Does NOT Scale

- Tic-tac-toe has thousands of possible states
- Chess has roughly 10^{47} possible positions
- Go has more than 10^{170} possible positions
- A giant system of equations becomes infeasible

© 2020, UNIVERSITY OF MARYLAND

Extensions to MDPs

- Infinite and continuous MDPs
- Partially observable MDPs (POMDPs)
- Undiscounted, average reward MDPs

© 2020, UNIVERSITY OF MARYLAND

POMDPs

- A Partially Observable Markov Decision Process is an MDP with hidden states. It is a hidden Markov model with actions.
- A POMDP is a tuple $\langle S, A, O, P, R, Z, \gamma \rangle$
  - S is a finite set of states, A is a finite set of actions
  - O is a finite set of observations
  - P is a state transition probability matrix
  - R is reward function
  - Z is observation function
  - γ is discount factor

© 2020, UNIVERSITY OF MARYLAND